

shukatsu_note コード解説資料

自分の書いたコードを理解するための資料。「①ディレクトリ・ファイルの役割」「②機能とコードの対応」「③TSX/Reactの文法」の3部構成。

第1部：全体の構造

Next.jsというフレームワークでは、`src/app/` 中のフォルダ構造そのものが、そのままURLの構造になるというルールがあります。

<code>src/app/page.tsx</code>	→ <code>https://.../</code>	(トップページ)
<code>src/app/notes/[category]/page.tsx</code>	→ <code>https://.../notes/interview</code>	
<code>src/app/companies/new/page.tsx</code>	→ <code>https://.../companies/new</code>	
<code>src/app/companies/[id]/page.tsx</code>	→ <code>https://.../companies/123</code>	

`[id]` や `[category]` のように角カッコが付いたフォルダ名は「ここは数字や文字列が何でも入る」という意味です（動的ルーティングと呼ばれます）。`companies/123` にアクセスすると、`page.tsx` の中で `123` という値を `params` として受け取れます。

`page.tsx` という名前のファイルだけが「画面」として扱われます。同じフォルダの中の他のファイル（`EntrySheetCard.tsx` など）は、その画面の中で使う「部品」です。

第2部：機能ごとのファイル一覧

データの土台

ファイル	役割
<code>prisma/schema.prisma</code>	DBの設計図。「企業」「ES」「リマインダー」などのテーブルの形を定義している
<code>prisma.config.ts</code>	どのDBに接続するかの設定（マイグレーション用）
<code>src/lib/prisma.ts</code>	アプリの実行時にDBとやり取りする窓口（1つだけ作って使い回す）
<code>src/lib/dateFormat.ts</code>	日付・時刻の表示形式を整える関数だけを集めたファイル
<code>src/lib/groupReminders.ts</code>	リマインダーを「今日/明日/日付ごと」に仕分けるロジックだけのファイル（画面表示とは無関係）
<code>src/lib/clickToCaret.ts</code>	クリックした位置が文字の何文字目かを計算する関数
<code>src/app/actions.ts</code>	「保存する」「削除する」など、DBを書き換える処理を全部集めた場所（Server Actions）

ログイン・認証

ファイル	役割
<code>src/auth.ts</code>	Auth.js（ログイン機能）の設定本体。Googleログインの設定もここ
<code>src/app/api/auth/[...nextauth]/route.ts</code>	Googleとログインのやり取りをする窓口
<code>src/lib/auth-helpers.ts</code>	「ログイン中か確認し、してなければログイン画面に飛ばす」処理をまとめた部品
<code>src/app/AuthButton.tsx</code>	右上の「Googleでログイン／ログアウト」ボタン
<code>src/types/next-auth.d.ts</code>	TypeScriptに「セッションにはuser.idという項目がある」と教えるための型定義

トップ画面

ファイル	役割
<code>src/app/page.tsx</code>	トップ画面本体。企業一覧・リマインダーボードを並べている「親」

src/app/loading.tsx	トップ画面に移動した瞬間、データが届くまでの間だけ出る簡易画面
src/app/ReminderBoard.tsx	全企業分のリマインダーを日付ごとにまとめて並べる部品
src/app/ReminderBoardItem.tsx	↑の中の、リマインダー1件分の表示・編集・チェック
src/app/HomeAddReminderForm.tsx	トップ画面の「+リマインダーを追加」フォーム（企業／個人を選べる）
src/app/DeleteCompanyButton.tsx	企業カードの削除ボタン（確認ダイアログ付き）
src/app/UndoBanner.tsx	「削除しました・元に戻す」のバナー
src/app/CopyButton.tsx	クリックした値をコピーする、使い回し用の小さいボタン

企業詳細ページ

ファイル	役割
src/app/companies/new/page.tsx	企業の新規登録フォーム画面
src/app/companies/[id]/page.tsx	企業詳細ページ本体。DBから企業データを取得し、下記の部品を並べる「親」
src/app/companies/[id]/loading.tsx	企業ページ用の簡易読み込み画面
src/app/companies/[id]/BackButton.tsx	「← 戻る」ボタン（編集集中なら確認を挟む）
src/app/companies/[id]/EditGuardContext.tsx	「今どこかが編集集中か」をページ全体で共有する仕組み
src/app/companies/[id]/MypageField.tsx	マイページURL/ID/PWの、1行分の表示・編集部品
src/app/companies/[id]/CompanyTabs.tsx	「メモ／ES／面接／インターン」タブの切り替え本体

リマインダー（企業ページ内）

ファイル	役割
ReminderRow.tsx	リマインダー1件の表示・編集（タイトル/日付/メモをまとめて編集）
AddReminderForm.tsx	「+リマインダーを追加」ボタンとフォーム

ES（設問・回答）

ファイル	役割
EntrySheetCard.tsx	ES1件の表示・編集（文字数カウント付き）
AddEntrySheetForm.tsx	「+設問を追加」ボタンとフォーム

面接

ファイル	役割
InterviewCard.tsx	面接記録1件の表示・編集
AddInterviewForm.tsx	「+面接記録を追加」ボタンとフォーム
InterviewTab.tsx	面接タブ全体（一覧+追加フォームをまとめる）

インターン・企業メモ

ファイル	役割
InternNoteCard.tsx / AddInternNoteForm.tsx / InternTab.tsx	インターン用の自由メモ（構造はES/面接と同じ考え方）
MemoEntryCard.tsx / AddMemoEntryForm.tsx / CompanyMemoTab.tsx	企業メモ（内容が空になったら自動削除される点が少し特殊）

個人メモ（自己PR・面接・GD）

--	--

ファイル	役割
src/app/notes/[category]/page.tsx	自己PR/面接/GD、3種類のページを1つのファイルで兼ねている
src/app/notes/NoteCard.tsx	メモ1件の表示・編集
src/app/notes/AddNoteForm.tsx	「+メモを追加」 ボタンとフォーム

第3部：この2つのパターンを理解すれば全部読める

このアプリのファイルは、ほとんどが次の2パターンの組み合わせでできています。

パターンA：「表示するだけのページ」 （Server Component）

```
export default async function Home() {
  const companies = await prisma.company.findMany({ ... });
  return <div>{companies.map(c => <p>{c.name}</p>)}</div>;
}
```

`async function` になっていて、`"use client"` が付いていないファイルがこれです。サーバー側だけで動き、直接DBに問い合わせで画面を作ります。ブラウザ側のクリック処理（`onClick` など）は書けません。

パターンB：「クリックで反応する部品」 （Client Component）

```
"use client";
import { useState } from "react";

export default function EntrySheetCard() {
  const [isEditing, setIsEditing] = useState(false);
  return <button onClick={() => setIsEditing(true)}>編集</button>;
}
```

ファイルの一番上に `"use client"` と書いてあるものがこれです。`useState` でボタンを押した後の状態を覚えておき、`onClick` でクリックに反応できます。その代わり、DBに直接問い合わせることはできません（保存処理は `actions.ts` から借りてきて使っています）。

第4部：TSX/JSXの文法解説

TSXは「TypeScript（JSの仲間）の中に、HTMLのようなものを直接書けるようにしたもの」です。1行ずつ意味を説明します。

HTMLタグの基本

```
<div className="p-4">
  <p className="font-bold">こんにちは</p>
</div>
```

- `<div>` … 意味を持たない「箱」。何かをグループ化してまとめる時に使う
- `<p>` … 文章（paragraph）
- `<span>` … 文章の中の、一部分だけを囲む小さい箱（改行されない `<div>` のようなもの）
- `<h1>` `<h2>` `<h3>` … 見出し（数字が小さいほど大きい見出し）
- `<button>` … 押せるボタン
- `<a href="...">` … リンク。`href` の中身が飛び先のURL
- `<input>` … 1行の入力欄
- `<textarea>` … 複数行の入力欄
- `<form action={...}>` … 入力欄をまとめて送信する枠。`action` に「送信先の関数」を指定する

className について

普通のHTMLでは `class="..."` と書きますが、TSXでは `className="..."` と書きます（`class` はJavaScriptで別の意味を持つ単語のため、区別しています）。中身のTailwindのクラス名（`text-sm` など）自体はどちらも同じです。

{ }（波カッコ）の意味

TSXの中で `{ }` を書くと、「ここはJavaScriptの値です」という合図になります。

```
<p>{company.name}</p>
```

これは「`company.name` という変数の中身をそのまま表示する」という意味です。`{ }` が無いと、ただの文字列 `"company.name"` として表示されてしまいます。

href と onClick の違い

- `<a href="/companies/1">`（または `<Link href="...">`）… ページそのものを移動する
- `<button onClick={() => ...}>` … ページは移動せず、ブラウザ内で何かの処理だけを実行する

ボタンなのにページ移動させたい時は `<Link>` を、その場で何か処理したいだけなら `<button onClick>` を使う、と使い分けています。

Props（プロップス）とは

```
<EntrySheetCard es={entry} companyId={5} />
```

これは、`EntrySheetCard` という部品に対して「`es` という名前で `entry` の中身を渡す」「`companyId` という名前で `5` を渡す」という意味です。渡された側は、こう受け取ります。

```
export default function EntrySheetCard({ es, companyId }: { es: EntrySheet; companyId: number }) {
```

`{ es, companyId }` の部分が「受け取る名前」、`: { es: EntrySheet; companyId: number }` の部分が「型（それぞれ何のデータか）」の指定です。TypeScriptはこの型指定のおかげで、「`es` に数字を渡そうとしている」のような間違いを保存前に教えてくれます。

useState

```
const [isEditing, setIsEditing] = useState(false);
```

「`isEditing` という名前の、今の状態を覚えておく箱」を作っています。最初の値は `false` です。`setIsEditing(true)` を呼ぶと、`isEditing` の中身が `true` に変わり、**画面が自動的に再描画されます**（これがReactの一番の特徴です）。

アロー関数

```
const handleClick = () => {
  console.log("押された");
};
```

`function handleClick() { ... }` と書くのと同じ意味の、短い書き方です。`onClick={() => setIsEditing(true)}` のように、その場で「名前を付けずに関数を書く」ためによく使います。

三項演算子（if/elseの短い書き方）

```
{reminder.completed ? "完了" : "未完了"}
```

「もし `reminder.completed` が `true` なら"完了"、`false` なら"未完了"を表示」という意味です。`条件 ? Aの場合 : Bの場合` という形です。

&&（条件付き表示）

```
{company.mypageUrl && <a href={company.mypageUrl}>マイページ</a>}
```

「もし `company.mypageUrl` に値があれば、右側の `<a>` タグを表示する」という意味です。値が無い（`null` や空文字）場合は何も表示されません。

?.（オプショナルチェイニング）と ??（null合体演算子）

```
item.company?.name
```

`item.company` が `null` かもしれない時に、そのまま `.name` を読むとエラーになります。`?.` を使うと「`company` が `null` なら、そこで止まって全体を `undefined` にする（エラーにしない）」という安全な読み方になります。

```
value={es.memo ?? ""}
```

「`es.memo` が `null` だったら、代わりに空文字 `""` を使う」という意味です。入力欄の初期値に `null` をそのまま渡すとエラーになるため、よく使っています。

.map()（一覧表示）

```
{companies.map((company) => (  
  <div key={company.id}>{company.name}</div>  
))}
```

配列（`companies` という複数件のデータのリスト）の中身を、1件ずつ `<div>` に変換して並べています。`key={company.id}` は、Reactが「どれがどれか」を見分けるために必須の指定です（重複しないID的な値を必ず指定します）。

TypeScriptの型がよく出るパターン

```
question: string          // 必ず文字列が入る  
answer: string | null     // 文字列、または「無い(null)」もありえる  
priority: number          // 数字  
completed: boolean        // true か false
```

| `null` が付いている項目は、DBで `String?`（末尾にクエスチョンマーク）になっているものと対応しています。

これを見ながら、実際のファイルを1つずつ照らし合わせてみると、理解が繋がりやすいと思います。分からない箇所が具体的に出てきたら、そのコードを見せてもらえれば、その部分だけ詳しく説明します。